CHARTER

Identify capabilities and areas of potential instability of the “rest api todo list manager”.

Identify documented and undocumented “rest api todo list manager” capabilities.

For each capability, create a script or small program to demonstrate the capability.

Exercise each capability identified with data typical to the intended use of the application.

---

runTodoManagerRestAPI-1.5.5.jar

Area: Exploratory testing for the /projects API endpoints

Environment: Ubuntu Linux

PC Model: Dell XPS 15 7590

START AND END TIME: 6:26PM to 7:11PM - 10/10/2026

TESTER

Shahzoab Shafi - ID: 261160765, E-mail: shahzoab.shafi@mail.mcgill.ca

TASK BREAKDOWN

DURATION: 45 minutes

SESSION NOTES

(1) GET /projects works as expected without any URL parameters. It returns 200 OK with the single default project (id 1, "Office Work") and its tasks relationship to todos 1 and 2. We note that the boolean fields completed and active are returned as strings ("false") rather than JSON booleans.
(2) GET /projects with active=false as URL parameter works as expected. It returns 200 OK with project 1, whose active field is "false".
(3) GET /projects with the URL encoded title "Office%20Work" as URL parameter works as expected and returns project 1.
(4) GET /projects with an unknown URL parameter (randomParam=x) returns 200 OK with every project. The unknown parameter is silently ignored. This is undocumented and matches the behaviour seen on /todos.
(5) HEAD /projects works as expected. It returns 200 OK with headers only and no body.
(6) POST /projects with all fields specified (title "School", completed false, active true, description "ECSE 429") works as expected. It returns 201 Created, a Location header "projects/2" and the new project with id 2.
(7) POST /projects with an empty body {} returns 201 Created and creates project 3 with an empty title. This is not expected: POST /todos rejects a missing title with 400, but projects accept it.
(8) POST /projects with an empty title "" returns 201 Created and creates project 4. This is not expected: todos reject an empty title with "title : can not be empty", while projects apply no title validation at all.
(9) POST /projects with a specified ID returns a 400 error code as expected, with the message "Invalid Creation: Failed Validation: Not allowed to create with id".
(10) POST /projects with active set to the string "yes" returns a 400 error code as expected, with the message "Failed Validation: active should be BOOLEAN". The boolean type is validated.
(11) POST /projects with malformed JSON returns a 400 error code as expected. The error message "java.io.EOFException: End of input at line 1 column 18 path $." is a raw Java exception and could be more specific. We also notice that the Content-Type header of the response is text/html although the body is JSON.
(12) POST /projects with an XML body works as expected and returns 201 Created. However, the new project received id 6 and id 5 was skipped. Repeating each rejected request one at a time showed that only the request rejected for a non-boolean active field (note 10) consumes an id, even though nothing is created.

(13) POST /projects with a malformed XML body returns a 400 error code as expected, with the message "Unclosed tag title at 23 [character 24 line 1]". The Content-Type header is again text/html for a JSON body.

(14) PUT /projects and DELETE /projects return 405 Method Not Allowed as expected.

(15) OPTIONS /projects shows the available endpoint methods as expected (Allow: OPTIONS, GET, HEAD, POST). OPTIONS is not documented but works.

(16) GET /projects/:id works as expected and returns project 1 with a 200 success code. We note that the single instance is wrapped in a "projects" array.

(17) GET /projects/:id works as expected when expecting an XML response (Accept: application/xml). It returns 200 OK with an XML body.

(18) GET /projects/:id with a non-existent ID returns 404 Not Found as expected, with the message "Could not find an instance with projects/999999".

(19) HEAD /projects/:id works as expected and returns 200 OK with headers only.

(20) POST /projects/:id with only a description works as expected. It returns 200 OK, amends the description and keeps the title, completed, active and the tasks relationship.

(21) PUT /projects/:id with only a title returns 200 OK and renames the project, but it also resets the description to an empty string (the value set in note 20 is lost) and the tasks relationship disappears from the response. The documentation describes PUT as amending the project, but it behaves like a full replacement that also drops relationships.

(22) GET /projects/:id after the PUT confirms the data loss: description is "" and the project no longer has any tasks. Project 1 lost its links to todos 1 and 2.



Figure 1: Reproduction of notes 20–22 on a restarted server (11/10/2026). The PUT with only a title resets the description and removes the tasks relationship.

(23) PATCH /projects/:id returns 405 Method Not Allowed as expected.
(24) DELETE /projects/:id on project 2 works as expected and returns 200 OK. Trying to DELETE the same project again returns 404 Not Found as expected, with the message "Could not find any instances with projects/2".
(25) GET /todos after deleting a project shows that todos 1 and 2 still exist, so deleting a project does not delete todos, as expected. It also confirms that todos 1 and 2 no longer have a tasksof relationship after the PUT in note 21.
(26) GET /projects/:id/tasks for project 1 returns 200 OK with an empty list, because the PUT in note 21 removed the relationships.
(27) GET /projects/:id/tasks with a non-existent project ID returns 200 OK with an empty list instead of the expected 404 Not Found. This is the same defect as bug 9 of the todos bug list.
(28) HEAD /projects/:id/tasks works as expected and returns 200 OK.
(29) POST /projects/:id/tasks with the id of todo 1 returns 201 Created and re-creates the relationship. The following GET /todos shows todo 1 with tasksof [1], so the relationship is visible from both sides as expected.
(30) POST /projects/:id/tasks with the id of a non-existent todo returns 404 Not Found as expected, but the response body is empty and gives no error message.
(31) POST /projects/:id/tasks with an empty body {} returns 400 with the message "title : field is mandatory". The message shows that the server tried to create a new todo from the body instead of rejecting the missing id. This is inconsistent with POST /todos/:id/tasksof {}, which silently creates a new empty project. GET /todos confirms that no todo was created here.
(32) DELETE /projects/:id/tasks/:id on project 1 and todo 1 works as expected and returns 200 OK. GET /todos/1/tasksof then returns an empty list, so the relationship is removed on both sides.
(33) Trying to DELETE /projects/:id/tasks/:id again for the relationship that we already deleted returns 404 Not Found as expected.
(34) PUT /projects/:id/tasks returns 405 Method Not Allowed as expected.
(35) GET /projects/:id/tasks/:id returns 404 Not Found with an empty text/html body. Since only DELETE is supported on this URL, we would expect 405 Method Not Allowed. This is the same defect as bug 5 of the todos session.
(36) GET /projects/:id/categories works as expected for project 1 and returns 200 OK with an empty collection.
(37) GET /projects/:id/categories with a non-existent project ID returns 200 OK with an empty collection instead of the expected 404 Not Found.
(38) POST /projects/:id/categories with category 1 returns 201 Created as expected. Sending exactly the same command again also returns 201 Created; a

check with GET /projects/1/categories shows that no duplicate relationship was created, as expected.

(39) GET /categories/:id/projects for category 1 returns 200 OK but does not list project 1, even though the project was just linked to it. The project/category relationship is only visible from the project side, unlike the project/todo relationship in notes 29 and 32.

(40) DELETE /projects/:id/categories/:id works as expected and returns 200 OK. Trying to DELETE it again returns 404 Not Found as expected.

(41) We get creative and use POST /projects/:id/tasks with a title instead of an id. It returns 201 Created with Location "todos/4": a new todo is created and linked to project 1 in a single call, which GET /todos confirms. This capability is not documented, since the documentation only describes linking an existing todo by id.

(42) We try the PUT /projects/:id/tasks method, which returns 405 (note 34), but with the header X-HTTP-Method-Override: POST. We get 201 Created and the relationship to todo 2 is created. The method restriction can be bypassed on projects too, the same defect as bug 6 of the todos session.

(43) POST /projects with an array as the title (["a","b"]) returns 201 Created and stores only the last element: project 7 has title "b". The same truncation as bug 1 of the todos session also affects projects.

Program:

The commands that were run for each note, numbered to match the session notes above. The server was restarted with its default data before note 1.

1. curl -i http://localhost:4567/projects
2. curl -i "http://localhost:4567/projects?active=false"
3. curl -i "http://localhost:4567/projects?title=Office%20Work"
4. curl -i "http://localhost:4567/projects?randomParam=x"
5. curl -I http://localhost:4567/projects
6. curl -i -X POST http://localhost:4567/projects \
   -H "Content-Type: application/json" \
   -d '{"title":"School","completed":false,"active":true,"description":"ECSE 429"}'
7. curl -i -X POST http://localhost:4567/projects \
   -H "Content-Type: application/json" \
   -d '{}'
8. curl -i -X POST http://localhost:4567/projects \
   -H "Content-Type: application/json" \

```
    -d '{"title":""}'
9.  curl -i -X POST http://localhost:4567/projects \
    -H "Content-Type: application/json" \
    -d '{"id":"99","title":"with id"}'
10. curl -i -X POST http://localhost:4567/projects \
    -H "Content-Type: application/json" \
    -d '{"title":"bad","active":"yes"}'
11. curl -i -X POST http://localhost:4567/projects \
    -H "Content-Type: application/json" \
    -d '{"title":"Broken"'
12. curl -i -X POST http://localhost:4567/projects \
    -H "Content-Type: application/xml" \
    -d '<project><title>XML proj</title></project>'
13. curl -i -X POST http://localhost:4567/projects \
    -H "Content-Type: application/xml" \
    -d '<project><title>broken'
14. curl -i -X PUT http://localhost:4567/projects
    curl -i -X DELETE http://localhost:4567/projects
15. curl -i -X OPTIONS http://localhost:4567/projects
16. curl -i http://localhost:4567/projects/1
17. curl -i http://localhost:4567/projects/1 \
    -H "Accept: application/xml"
18. curl -i http://localhost:4567/projects/999999
19. curl -I http://localhost:4567/projects/1
20. curl -i -X POST http://localhost:4567/projects/1 \
    -H "Content-Type: application/json" \
    -d '{"description":"amended with POST"}'
21. curl -i -X PUT http://localhost:4567/projects/1 \
    -H "Content-Type: application/json" \
    -d '{"title":"Renamed"}'
22. curl -i http://localhost:4567/projects/1
23. curl -i -X PATCH http://localhost:4567/projects/1 \
    -H "Content-Type: application/json" \
    -d '{"title":"x"}'
24. curl -i -X DELETE http://localhost:4567/projects/2
    curl -i -X DELETE http://localhost:4567/projects/2
25. curl -i http://localhost:4567/todos
26. curl -i http://localhost:4567/projects/1/tasks
```

27. curl -i http://localhost:4567/projects/999999/tasks
28. curl -I http://localhost:4567/projects/1/tasks
29. curl -i -X POST http://localhost:4567/projects/1/tasks \
    -H "Content-Type: application/json" \
    -d '{"id":"1"}'
    curl -i http://localhost:4567/todos
30. curl -i -X POST http://localhost:4567/projects/1/tasks \
    -H "Content-Type: application/json" \
    -d '{"id":"999"}'
31. curl -i -X POST http://localhost:4567/projects/1/tasks \
    -H "Content-Type: application/json" \
    -d '{}'
    curl -i http://localhost:4567/todos
32. curl -i -X DELETE http://localhost:4567/projects/1/tasks/1
    curl -i http://localhost:4567/todos/1/tasksof
33. curl -i -X DELETE http://localhost:4567/projects/1/tasks/1
34. curl -i -X PUT http://localhost:4567/projects/1/tasks
35. curl -i http://localhost:4567/projects/1/tasks/2
36. curl -i http://localhost:4567/projects/1/categories
37. curl -i http://localhost:4567/projects/999999/categories
38. (2 times) curl -i -X POST http://localhost:4567/projects/1/categories \
    -H "Content-Type: application/json" \
    -d '{"id":"1"}'
    curl -i http://localhost:4567/projects/1/categories
39. curl -i http://localhost:4567/categories/1/projects
40. (2 times) curl -i -X DELETE http://localhost:4567/projects/1/categories/1
41. curl -i -X POST http://localhost:4567/projects/1/tasks \
    -H "Content-Type: application/json" \
    -d '{"title":"made via project"}'
    curl -i http://localhost:4567/todos
42. curl -i -X PUT http://localhost:4567/projects/1/tasks \
    -H "X-HTTP-Method-Override: POST" \
    -H "Content-Type: application/json" \
    -d '{"id":"2"}'
43. curl -i -X POST http://localhost:4567/projects \
    -H "Content-Type: application/json" \
    -d '{"title":["a","b"]}'

Areas not covered:

- HEAD and OPTIONS on /projects/:id/categories and /projects/:id/tasks/:id
- XML request bodies for PUT /projects/:id and for the relationship endpoints
- Filtering GET /projects on completed and description, and combining several filters
- X-HTTP-Method-Override on every other restricted project endpoint (only /projects/:id/tasks was tried)
- Interaction between projects and categories beyond a single link (categories created through a project, deleting a category that is linked to a project)

Bugs logged

- Bug #11: PUT /projects/:id with only a title resets the description and removes the project's relationships to its todos (notes 21, 22, 25).
- Bug #12: POST /projects accepts a missing or empty title and creates a project with an empty title, while todos reject the same input with 400 (notes 7, 8).
- Bug #13: GET /projects/:id/categories with a non-existent project ID returns 200 OK with an empty list instead of 404 Not Found (note 37).
- Bug #14: A project linked to a category with POST /projects/:id/categories is not listed by GET /categories/:id/projects; the relationship is one-sided (note 39).
- Bug #15: The X-HTTP-Method-Override: POST header turns a forbidden PUT /projects/:id/tasks into a POST and creates the relationship with 201 Created (note 42; same defect as bug 6).
- Bug #16: POST /projects with an array as the title silently keeps only the last element (note 43; same defect as bug 1).
- Bug #17: GET /projects/:id/tasks/:id returns 404 Not Found instead of 405 Method Not Allowed (note 35; same defect as bug 5).
- Bug #18: A POST /projects rejected for an invalid boolean field still consumes an id, leaving a gap in project ids (note 12).
- Bug #19: Error responses for malformed JSON and XML have the Content-Type header text/html although their body is JSON (notes 11, 13).
- Already logged: GET /projects/:id/tasks with a non-existent project ID returns 200 OK instead of 404 (note 27), which is bug 9 of the todos bug list.

Other Session Ideas

- Check whether PUT /todos/:id also removes category links and whether PUT /categories/:id removes project links, to see if the relationship loss in bug 11 affects every entity.
- Explore creating todos through POST /projects/:id/tasks with a body (note 41) further: which fields are accepted and whether validation matches POST /todos.
- Compare the validation rules of projects, todos and categories field by field (required title, empty title, boolean types).
- Check bidirectionality of every relationship: project/category, category/todo and todo/project.
- Track ids across many rejected requests to see which validation failures consume an id.

Concerns and Areas of Potential Risk

- **Silent data loss on update:** PUT /projects/:id with a partial body wipes fields and deletes relationships with a 200 success code, so clients never learn that data was lost.
- **Missing input validation:** projects can be created with no title or an empty title, and an array title is silently truncated.
- **One-sided relationships:** a project linked to a category is not visible from the category, so the two sides of the data disagree.
- **Method override:** X-HTTP-Method-Override bypasses method restrictions on project endpoints as well as todo endpoints, which may defeat firewalls or authorization rules that filter on the HTTP method.
- **Inconsistent error handling:** non-existent projects return 200 on relationship endpoints, unsupported methods return 404 instead of 405, and some errors come back with an empty body or a text/html content type.

Capabilities (the note numbers match the labels in session/projects_session.sh)

- CRUD operations on the PROJECT entity in JSON and XML: create (notes 6, 12), read (notes 1, 16, 17), amend with POST and PUT (notes 20–22), delete (note 24), and rejection of invalid input (notes 9–11, 13, 14, 18, 23).
- HEAD and OPTIONS operations on /projects, /projects/:id and /projects/:id/tasks (notes 5, 15, 19, 28).
- Filtering GET /projects with URL query parameters such as title and active (notes 2–4).

- Querying, creating and deleting PROJECT/TODO relationships (notes 26–35).
- Querying, creating and deleting PROJECT/CATEGORY relationships (notes 36–40).
- Undocumented: creating a new todo already linked to a project with POST /projects/:id/tasks and a todo body (note 41).
- Request method overriding with X-HTTP-Method-Override on relationship endpoints, which is a defect (note 42, bug 15).

Summary of Session Findings

- Project CRUD works in JSON and XML: 201 with a Location header on create, 404 for unknown ids, 405 for unsupported methods on /projects and /projects/:id.
- Projects validate less than todos: no title validation and an array title is truncated, although boolean types are checked.
- PUT /projects/:id behaves as a full replacement and drops the project's task links, so data is lost with a 200 response.
- Project/todo links are visible from both sides; project/category links are visible from the project side only.
- Relationship endpoints do not check that the parent project exists (they return 200 for an unknown project).
- Undocumented behaviour: POST /projects/:id/tasks with a todo body creates a new todo already linked to the project. OPTIONS works, and unknown query parameters are ignored.
- Error handling is inconsistent: 404 instead of 405, some error responses have an empty body, and parse errors come back as text/html.
- Bugs 1, 5, 6 and 9 from the todos bug list also affect projects, which suggests the two entities share the same implementation.

Decisions Made

- Started from a freshly restarted server with default data and went endpoint by endpoint in the order of /docs: /projects, /projects/:id, /tasks, /categories.
- Used curl -i for every request so the status code and headers were recorded.

- When id 5 was skipped (note 12), repeated each rejected request on its own to find which one consumed the id.
- After every change, checked the result with GET from both sides (notes 22, 25, 32, 39).
- After the PUT removed the links (note 21), re-linked todo 1 (note 29) so the relationship tests could continue.
- Tested categories only through their link with projects, because a two-person team covers todos and projects.
- Tried a todo body instead of an id (note 41) and the method override header (note 42), based on findings from the todos session.
- Numbered the bugs from 11 to continue the team's bug list.
- Reproduced notes 20–22 on a restarted server to capture Figure 1.

Files Used and Created

- Used: runTodoManagerRestAPI-1.5.5.jar, http://localhost:4567/docs
- session/projects_session.sh: replays every command of this session, labelled by note number, and demonstrates each capability with typical data
- session/figure1_put_data_loss.png: screenshot for Figure 1
- Bug_Summary_Reports_Projects.docx: bugs #11–#19
- Unit tests that came from this session: ProjectsTest, KnownBugsTest (bugs 11–19), UndocumentedApiTest, CommandLineQueryTest